

Audio Lyrics Aligner: Detailed Runtime Flow Walkthrough

This document explains the code as it exists today. It focuses on the working runtime path, not the full future product vision. The current codebase is an audio ingestion and diagnostics pipeline. It does not yet perform feature extraction, alignment, slide resolution, or OSC output.

Current truth: the code already captures or simulates audio, pushes chunks through a bounded queue, computes basic signal diagnostics, and logs metrics. The “lyrics alignment” parts are mostly planned types and interfaces, not runtime behavior yet.

1. Fast Summary

```
CLI args + env vars
  |
  v
AppConfig
  |
  v
main.py chooses AudioSource
  |
  +--> SimulatedAudioSource
  |      or
  +--> MicrophoneAudioSource
  |
  v
source.chunks() yields AudioChunk
  |
  v
AudioIngestionRuntime producer thread
  |
  v
BoundedAudioQueue
  |
```

```
    v
AudioIngestionRuntime consumer loop
    |
    v
compute RMS / peak / silence / clipping
    |
    v
RuntimeMetrics + log output + RuntimeReport
```

2. Read Order

1. [src/lyrics_aligner/main.py](#)
2. [src/lyrics_aligner/config.py](#)
3. [src/lyrics_aligner/ports/audio_source.py](#)
4. [src/lyrics_aligner/domain/models.py](#)
5. [src/lyrics_aligner/application/runtime.py](#)
6. [src/lyrics_aligner/application/metrics.py](#)
7. [src/lyrics_aligner/adapters/audio/simulated.py](#)
8. [src/lyrics_aligner/adapters/audio/microphone.py](#)

3. Package-Level Mental Model

domain

Pure data structures and future business concepts. Today the main active domain type is `AudioChunk`.

ports

Abstract contracts. The runtime depends on `AudioSource`, not on a concrete adapter.

adapters

Concrete implementations. Right now there are two audio adapters: `simulated` and `microphone`.

application

Orchestration and lifecycle. This is where threading, queueing, metrics, and logging live.

4. Entry Point: What main.py Actually Does

main.py is intentionally thin. It does not contain signal-processing logic. It acts as a bootstrapping layer.

Main responsibilities:

- parse CLI arguments
- load environment-driven configuration
- construct the selected audio source
- construct the runtime
- run the runtime and log the final summary

The most important helper in the file is `_build_audio_source(config)`. That function decides whether the program runs in simulated mode or microphone mode.

Startup Sequence

1. `_parse_args()` reads CLI flags.
2. `_config_from_args()` merges CLI args with env-based defaults.
3. `_build_audio_source()` returns an `AudioSource`.
4. `AudioIngestionRuntime(...)` is created.
5. `runtime.run()` starts producer/consumer processing.
6. A final summary log line is emitted from the returned report.

5. Configuration Flow

Configuration is centralized in [src/lyrics_aligner/config.py](#).

Why AppConfig Exists

The code does not scatter constants everywhere. Instead, it keeps runtime choices in one dataclass. That makes the boot path deterministic and easy to extend later.

Important Fields

- `audio_source`: selects `simulated` or `microphone`
- `sample_rate`: audio sampling frequency
- `channels`: currently expected to be `mono` for the main path
- `block_size`: samples per chunk
- `audio_queue_capacity`: bounded queue size in the runtime
- `input_device`: microphone device name or index
- `simulation_duration_seconds`: simulated stream length
- `diagnostics_interval_seconds`: log cadence
- `silence_threshold_rms`: threshold for silent chunk counting

- `clipping_threshold_peak`: threshold for clipped chunk counting

Env Var Precedence

`AppConfig.from_env()` reads `LYRICS_ALIGNER_*` variables. Then `_config_from_args()` lets CLI flags override some of those values. That means the effective precedence is:

```
hardcoded defaults
  -> environment variables
  -> CLI arguments
```

6. The Port Abstraction

The key interface is [src/lyrics_aligner/ports/audio_source.py](#).

```
class AudioSource(Protocol):
    def chunks(self) -> Iterator[AudioChunk]: ...
```

That single interface is what keeps the runtime generic. The application layer only knows “I can iterate audio chunks.” It does not know whether those chunks came from NumPy synthesis, a microphone callback, a WAV file, or some future network source.

7. Core Domain Model: AudioChunk

The main active domain model is [src/lyrics_aligner/domain/models.py](#).

```
@dataclass(frozen=True, slots=True)
class AudioChunk:
    samples: FloatArray
    captured_at: float
    sequence_number: int
```

Each field matters:

- `samples`: the raw waveform for that time slice
- `captured_at`: when the chunk was considered captured
- `sequence_number`: monotonic chunk ordering

Most of the runtime is effectively “move `AudioChunk` safely from source to processor.”

8. Simulated Audio Adapter

[adapters/audio/simulated.py](#) is the easier adapter to understand because it does not involve callback APIs.

What It Does

- validates a sine-wave configuration
- computes chunks of waveform samples
- paces delivery in real time
- yields `AudioChunk` objects

Why It Matters

It lets the rest of the pipeline be tested without touching real audio hardware. This is important because queue behavior, shutdown behavior, and metrics logic can all be built before device integration is stable.

Key Details

- `duration * sample_rate` determines total samples
- the loop walks those samples in `block_size` increments
- each block computes a deterministic sine wave
- the adapter waits until the scheduled capture time before yielding
- `stop()` interrupts the wait via a threading event

9. Microphone Audio Adapter

[adapters/audio/microphone.py](#) is more subtle because real audio APIs are callback-driven.

What It Does

- opens a `sounddevice.InputStream`
- receives audio frames inside a callback
- converts those frames to mono `float32`
- packages them into `AudioChunk`
- pushes them into an internal queue
- yields them to the runtime from `chunks()`

Why There Is an Internal Queue

The audio callback should stay fast. Doing heavier work inside that callback is risky because callback stalls can cause audio dropouts or timing issues. So the callback only does the minimum:

1. receive input samples

2. convert shape and dtype
3. wrap them in AudioChunk
4. put them into a queue

Then the outer iterator yields from that queue at normal Python speed.

Important Distinction

The microphone adapter has its own small internal queue, but the main backpressure policy for the application still lives in the runtime's bounded queue. So there are two queue layers with different purposes:

- adapter queue: callback-safe handoff
- runtime queue: application-level backpressure and drop policy

10. Runtime: The Actual Working Pipeline

`application/runtime.py` is the operational center of the codebase.

Two Logical Sides

Producer Side

Reads chunks from the source and inserts them into the bounded queue.

Consumer Side

Pulls chunks from the bounded queue, processes them, and logs metrics.

`run()`

`run()` creates a producer thread and then immediately starts the consumer loop on the main thread. This means the runtime is already a concurrent system even though the processing stage is still simple.

Producer Thread

`_produce()` iterates over `self._source.chunks()`. For every chunk:

1. check whether stop was requested
2. push into the bounded queue
3. increment `chunks_received`
4. increment `chunks_dropped` if the queue dropped one
5. update `queue_high_water_mark`

Consumer Loop

The main loop in `run()` does this repeatedly:

1. try to get a chunk from the bounded queue
2. if a chunk exists, process it
3. if the diagnostics interval has elapsed, log a snapshot
4. keep going until the producer is dead and the queue is empty

11. Bounded Queue and Backpressure

`BoundedAudioQueue` implements a deliberate real-time policy: when the queue is full, drop the oldest chunk instead of blocking the producer.

```
if len(self._items) == self._capacity:
    self._items.popleft()
    dropped = True
self._items.append(item)
```

That decision reflects the architecture document: stale live audio is usually less valuable than current live audio. This is why the queue is bounded at all. If it were unbounded, latency could grow silently.

Why Oldest Is Dropped

- the system stays near real time
- memory usage stays bounded
- capture is not blocked by downstream slowness
- metrics still reveal the overload via `chunks_dropped`

12. Processing Step Today

The current processing logic is intentionally small. It is not yet feature extraction or alignment.

In `_process(chunk)` the runtime computes:

- RMS energy from the waveform
- peak absolute value from the waveform

Then it classifies the chunk:

- silent if `rms <= silence_threshold_rms`
- clipped if `peak >= clipping_threshold_peak`

That is the entire signal-analysis stage at the moment. It exists to validate the runtime path before more complex feature extraction is added.

13. Metrics

`application/metrics.py` stores counters in a mutable dataclass.

Currently relevant counters:

- `chunks_received`
- `chunks_dropped`
- `silent_chunks`
- `clipped_chunks`
- `queue_high_water_mark`

Other counters are placeholders for future stages such as feature extraction, matching, and presentation output.

14. Logging and RuntimeReport

The runtime periodically emits a log line with the latest values. At the end, it returns a `RuntimeReport`.

```
device=...
rms=...
peak=...
queue=current/capacity
chunks_received=...
chunks_dropped=...
silent_chunks=...
clipped_chunks=...
```

`RuntimeReport` is effectively the final snapshot returned to `main()`. It contains:

- device name
- metrics snapshot
- last observed RMS
- last observed peak
- final queue size and capacity

15. Threading Model

The current runtime has a simple but important threading model:

- main thread: consumer loop + periodic logging
- producer thread: iterates source chunks and fills bounded queue

- optional source callback thread: inside sounddevice when microphone mode is active

In microphone mode this means there can effectively be three layers of execution:

1. sounddevice callback receives frames
2. microphone adapter iterator yields chunks
3. runtime producer thread forwards them into the bounded queue
4. main-thread consumer processes them

16. Shutdown Behavior

Shutdown is coordinated through events and queue closing.

- `AudioIngestionRuntime.stop()` sets a stop event
- the runtime queue is closed
- if the source has a `stop()` method, it is called
- the producer thread is joined before the final report returns

This is why both adapters implement their own way of interrupting waits: the simulated source wakes its event-based delay and the microphone source pushes a sentinel through its internal queue.

17. What the Code Does Not Yet Do

These planned concepts exist in shape but not in runtime behavior:

- `FeatureFrame`: no feature extractor uses it yet
- `MatchResult`: no alignment engine produces it yet
- `SlideCommand`: no presentation adapter sends it yet

So if you are trying to understand “where the lyrics alignment happens,” the honest answer is: it does not happen yet in the executable path. What exists today is the ingestion foundation that alignment will later plug into.

18. End-to-End Example: Simulated Mode

```
.venv/bin/python -m lyrics_aligner.main
```

1. main parses args
2. config resolves to `audio_source=simulated`
3. `SimulatedAudioSource` is created
4. runtime starts producer thread
5. simulated source yields timed sine-wave chunks

6. producer inserts them into BoundedAudioQueue
7. consumer computes RMS/peak on each chunk
8. periodic logs are emitted
9. final RuntimeReport is returned and logged

19. End-to-End Example: Microphone Mode

```
.venv/bin/python -m lyrics_aligner.main --audio-source microphone --input-device "Built-in Microphone"
```

1. main parses args
2. config resolves to audio_source=microphone
3. MicrophoneAudioSource is created
4. runtime starts producer thread
5. sounddevice callback receives live frames
6. callback converts frames to AudioChunk and enqueues them internally
7. microphone source iterator yields those chunks
8. runtime producer inserts them into the bounded queue
9. runtime consumer computes diagnostics and logs metrics

20. Best Way to Understand the Code in Practice

1. Run simulated mode first.
2. Read **main.py** top to bottom.
3. Read **runtime.py** next.
4. Then read **simulated.py**.
5. Only after that read **microphone.py**.

That order works because simulated mode is the simplest executable path. If you start with the microphone adapter, you end up learning callback plumbing before you understand the system around it.

21. Final Takeaway

The current code is a well-structured real-time ingestion skeleton:

- configuration comes in through config/env/CLI

- an adapter yields `AudioChunk` objects
- the runtime protects itself with a bounded queue
- processing currently means diagnostics, not alignment
- metrics and logging show whether the ingestion path behaves correctly

Once you understand that, the rest of the future system is easier to imagine: feature extraction will replace “compute RMS/peak only,” and alignment plus slide decisions will sit downstream of the same runtime backbone.